An Intelligent SIEM Framework for Real-Time Threat Detection and Automated Incident Response Using Machine Learning and Behavioural Analytics

Alex Muiruri

155361

A Project Proposal Submitted to School of Computing Engineering Sciences, Strathmore University for the Award of Bachelor of Science in Computer Networks and Cyber Security

STRATHMORE UNIVERSITY

NAIROBI, KENYA

June 2026

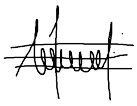
Declaration and Approval

I declare that this work has not been submitted or previously submitted and approved, in whole or partial, for the award of a degree by this or any other University. To the best of my knowledge and belief, the report contains no material previously published or written by another person except where due reference is made in the dissertation itself.

Student

Alex Muiruri

155361

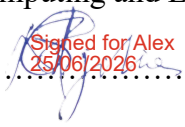


Signature ……. Date…….25 June 2026…….

Approval

Mr. James Gikera

School of Computing and Engineering Sciences



Signature ……. Date ……25 June 2026……

Acknowledgement

First and foremost, I would like to thank God for the strength, clarity, and perseverance to see this project through.

I am deeply grateful to my supervisor, Dr. James Gikera, for his guidance, patience, and constructive feedback throughout this project. His insights helped shape the direction and quality of this work significantly.

I would also like to thank the faculty and staff of the School of Computing and Engineering Sciences at Strathmore University for the knowledge and support provided throughout the course of this programme.

Finally, I acknowledge the open-source communities behind Wazuh, Cowrie, and the scikit-learn library, whose tools make security research at this level accessible to students and researchers worldwide.

Abstract

Modern organisations face an escalating volume of cyber threats that traditional security tools are ill-equipped to handle in isolation. Security teams are overwhelmed by high rates of false positives, limited network visibility, and slow manual incident response processes that leave critical systems exposed. This project proposes the design and implementation of an Integrated Cyber Threat Detection and Response Platform combining four complementary security layers into a unified, automated defence framework.

The platform integrates four layers: a Wazuh-based SIEM core for centralised log collection and real-time monitoring, a Cowrie honeypot for deception-based attacker capture, a Python machine learning engine using Random Forest and Isolation Forest models for threat classification, and an automated response module that executes containment playbooks upon detection. The details of each layer are discussed fully in the chapters that follow.

The integrated platform will be deployed and validated in a controlled virtual machine environment using VMware Workstation, with attack scenarios simulated using Kali Linux to measure detection accuracy, response time, and overall system performance. This project addresses a critical gap in affordable, unified threat detection for small and medium-sized enterprises and contributes a practical, open-source deployable framework for proactive cyber defence.

Keywords: SIEM, Honeypot, Threat Detection, Machine Learning, Anomaly Detection, Random Forest, Isolation Forest, Intrusion Detection, Python Automation, Cyber Security, Wazuh, Wazuh Dashboard, Wazuh Indexer, Cowrie, Threat Hunting, CICIDS2017, NSL-KDD.

Table of Contents

Declaration and Approval ii
Acknowledgement iii
Abstract iv
Table of Contents 1
List of Abbreviations and Acronyms 3
Definition of Terms 4
Chapter 1: Introduction 5
1.1 Background to the Project 5
1.2 Problem Statement 6
1.3 Objectives of the Study 8
1.3.1 General Objective 8
1.3.2 Specific Objectives 8
1.5 Significance of Study 10
Chapter 2: Literature Review 11
2.1 Introduction 11
2.2 Technology Review 11
2.2.1 Security Information and Event Management (Wazuh) 11
2.2.2 Honeypot Deception Technology (Cowrie) 12
2.2.3 Machine Learning for Intrusion Detection (Random Forest and Isolation Forest) 13
2.2.4 Intrusion Detection Datasets (CICIDS2017 and NSL-KDD) 13
2.2.5 Python-Based Security Automation and Orchestration 14
2.2.6 Virtualisation and Attack Simulation (VMware and Kali Linux) 15
2.3 Cyber Attack Vulnerability Analysis 15
2.3.1 Brute Force and Credential Stuffing Attacks 15
2.3.2 Port Scanning and Network Reconnaissance 16
2.3.3 Lateral Movement 16
2.3.4 Privilege Escalation 17
2.3.5 Data Exfiltration 17
2.3.6 Command and Control Beaconing 18
2.3.7 Suspicious Authentication Patterns 19
2.4 Existing Solutions for Integrated Threat Detection 19

2.4.1 Machine Learning: Algorithms, Real-World Applications and Research Directions.....20
2.4.2 A Survey of Honeypots and Honeynets for IoT and Cyber-Physical Systems 21
2.4.3 Network Intrusion Detection: A Systematic Study of Machine Learning and Deep Learning Approaches.....21
2.4.4 Existing Research Gap 22
2.5 Conceptual Framework 23
Chapter 3: Design Methodology.....25
3.1 Introduction 25
3.2 Project Methodology.....25
3.3 Feature and Design Requirements.....27
3.3.1 Functional Requirements.....27
3.3.2 Non-Functional Requirements.....28
3.4 Implementation Requirements.....28
3.4.1 Hardware Requirements 28
3.4.2 Software Requirements 28
3.5 Testing and Validation 29
3.5.1 Testing Strategy.....29
3.5.2 Test Environment 30
3.5.3 Attack Simulation Scenarios 30
3.5.4 Validation Metrics and Acceptance Criteria.....30
References.....32

List of Abbreviations and Acronyms

APT — Advanced Persistent Threat

C2 — Command and Control

CICIDS — Canadian Institute for Cybersecurity Intrusion Detection System

DNS — Domain Name System

DSR — Design Science Research

FTP — File Transfer Protocol

HTTP — Hypertext Transfer Protocol

IDS — Intrusion Detection System

IP — Internet Protocol

JSON — JavaScript Object Notation

ML — Machine Learning

MTTD — Mean Time to Detect

MTTR — Mean Time to Respond

NSL-KDD — Network Security Laboratory – Knowledge Discovery and Data mining

SIEM — Security Information and Event Management

SME — Small and Medium-sized Enterprise

SOAR — Security Orchestration, Automation and Response

SSH — Secure Shell

TCP — Transmission Control Protocol

VM — Virtual Machine

Definition of Terms

Anomaly Detection: A machine learning method that identifies events or data points deviating significantly from established normal behaviour, without requiring labelled examples of attacks for training.

Brute Force Attack: A method of gaining unauthorized access to a system by systematically attempting all possible username and password combinations until a valid pair is found.

Command and Control (C2) Beaconing: A pattern of network behaviour in which compromised host makes regular, periodic outbound connections to an attacker-controlled server at consistent time intervals to receive instructions or report status.

Honeypot: A deliberately exposed decoy system or service configured to appear as a legitimate target, designed to attract, log, and study attacker behaviour without exposing real infrastructure.

Intrusion Detection System (IDS): A system that monitors network traffic or host activity for signs of unauthorized access, policy violations, or malicious behaviour, and generates structured alerts when suspicious events are detected.

Lateral Movement: A post-compromise technique in which an attacker who has gained access to one system extends their reach across a network by authenticating to or exploiting adjacent hosts.

Machine Learning (ML): A branch of artificial intelligence in which algorithms learn statistical patterns from historical data to make predictions or classifications on new data without explicit rule-based programming.

Random Forest: A supervised machine learning algorithm that constructs an ensemble of decision trees during training and determines the final classification by majority vote across all trees.

Security Information and Event Management (SIEM): A platform that centralizes the collection, normalisation, and correlation of security event data from multiple network and endpoint sources, enabling real-time threat detection, alerting, and historical analysis.

Virtual Machine (VM): A software-based emulation of a physical computer that runs an operating system and applications in an isolated environment hosted on a physical machine, allowing multiple isolated systems to operate on a single hardware platform.

Chapter 1: Introduction

1.1 Background to the Project

Cyber attacks have grown considerably in frequency and technical sophistication over the past decade. Organizations across every sector, from financial institutions to hospitals to government agencies, now face persistent threats that range from automated scanning tools operated by amateur actors to coordinated intrusion campaigns carried out by state-sponsored groups. The financial and operational impact of security incidents has grown steadily, with delayed detection and slow response consistently identified as factors that increase the severity of breaches (Sarker, 2021).

Security Information and Event Management (SIEM) systems were developed to address this detection gap. A SIEM platform collects log data from across an organisation's network from servers, workstations, firewalls, and applications and correlates those events to surface potential threats. Early SIEM tools operated entirely on hand-written rules: if a certain number of failed logins occurred within a defined window, an alert fired. This approach works well against known, predictable attack patterns but struggles considerably against attackers who operate slowly, change their methods, or exploit behaviours that no rule anticipated. The result is a security operations environment defined by alert fatigue, where analysts process thousands of notifications daily, most of which are false positives, while the genuine threats risk being buried.

Machine learning has changed how intrusion detection systems approach this problem. Rather than relying on fixed detection rules, ML-based systems learn what normal network behaviour looks like and flag deviations from that baseline. Supervised classifiers such as Random Forest can identify known attack categories with high accuracy, while unsupervised methods such as Isolation Forest can detect novel threats that lack labelled training examples (Ahmad et al., 2021). Benchmark datasets such as CICIDS2017, produced by the Canadian Institute for Cybersecurity, provide a realistic foundation for training models against contemporary attack traffic including brute force, infiltration, and botnet activity (Khraisat & Alazab, 2021).

Honeypot technology offers a different but complementary approach. A honeypot is a decoy system configured to look like a legitimate target but designed specifically to attract, log, and study attacker behaviour. Unlike passive monitoring tools, honeypots generate threat intelligence with

no false positives, any interaction with a honeypot is by definition unauthorised. Tools such as Cowrie, an open-source medium-interaction honeypot, simulate SSH and Telnet services and capture every command an attacker executes during a session, providing detailed behavioural data that is difficult to obtain through passive log analysis alone (Franco et al., 2021).

Despite the availability of these individual technologies, most small and medium-sized enterprises (SMEs) in developing economies, including Kenya, operate without any structured threat detection infrastructure. Deploying commercial SIEM platforms or managed detection and response services requires budgets and technical personnel that most SMEs do not have. Open-source tools exist but are typically deployed in isolation, without the integration or automation needed to make them operationally useful. This project addresses that gap directly by designing a unified, four-layer platform that combines Wazuh-based SIEM, Cowrie honeypot deception, machine learning threat hunting, and Python-driven automated response into a single deployable framework accessible to resource-constrained organisations.

1.2 Problem Statement

Organisations face a growing mismatch between the volume of security events their networks generate and their capacity to analyse them. Conventional SIEM tools that rely on static, signature-based detection rules produce large volumes of false positive alerts, forcing security analysts to spend the majority of their time investigating benign events rather than responding to real threats. Research into security operations workloads has found that alert volume and false positive rates are the primary challenge for analysts, with most organisations lacking the capacity to investigate every generated notification (Vielberth et al., 2021).

Beyond alert fatigue, conventional tools have limited capability against attacks that do not match known signatures. Advanced Persistent Threats (APTs), which involve slow, deliberate intrusion campaigns that blend into normal network activity, are routinely missed by rule-based detection systems. Lateral movement between internal hosts, credential stuffing conducted over extended time periods, and command and control beaconing using low-frequency intervals are categories of attack that generate no alert in a standard SIEM configuration but represent serious, active compromises of organisational infrastructure.

A further problem is the absence of automated response capability in most deployed security systems. Even when a threat is detected, the process of containing it, blocking an attacking IP address, isolating a compromised account, or notifying an administrator typically requires manual intervention, introducing a delay during which damage continues. For SMEs without dedicated security operations staff available around the clock, this gap between detection and response is where the most harm occurs.

No currently available open-source solution integrates centralised log management, deception-based threat intelligence, machine learning detection, and automated response within a single, deployable platform. Existing research addresses these capabilities individually but does not provide a unified, operationally tested framework suitable for resource-constrained environments. This project addresses that specific gap.

1.3 Objectives of the Study

1.3.1 General Objective

The main objective of this project is to design, develop, and validate an Integrated Cyber Threat Detection and Response Platform that combines SIEM-based log monitoring, honeypot deception, machine learning threat detection, and automated Python-driven incident response into a unified, open-source deployable framework for small and medium-sized enterprises.

1.3.2 Specific Objectives

i. To investigate the execution methods and detection signatures of seven target attack categories through peer-reviewed literature producing a documented threat profile to guide platform design.

ii. To analyse a minimum of five peer-reviewed studies on SIEM monitoring, honeypot deployment, and machine learning intrusion detection, identifying the technical gaps that the proposed integrated platform addresses.

iii. To design and develop the four platform layers which will include Wazuh SIEM, Cowrie honeypot, a scikit-learn ML engine, and a Python automated response module, achieving full functional integration by the project implementation deadline.

iv. To test and validate the platform against five simulated attack scenarios using Kali Linux in VirtualBox, measuring detection accuracy, false positive rate, mean time to detect, and mean time to respond.

1.4 Scope and Limitations

This project covers the design, implementation, and laboratory testing of a four-layer integrated threat detection and response platform. The SIEM layer is scoped to Linux-based endpoint monitoring using Wazuh agents, covering authentication logs, system call logs, network connection events, and file integrity monitoring. The honeypot layer is limited to SSH and Telnet service emulation using Cowrie. The machine learning layer addresses seven attack categories: brute force, credential stuffing, port scanning, lateral movement, privilege escalation, data exfiltration, and command and control beaconing.

The automated response layer covers three response actions: firewall-based IP blocking, user account isolation, and email-based administrator notification.

The platform is tested exclusively in a controlled virtual machine environment and is not deployed to a live production network. Testing is therefore limited to simulated attack scenarios and does not capture the full range of organic traffic patterns present in operational enterprise environments. The machine learning models are trained on publicly available benchmark datasets; performance on real organisational traffic may vary and would require retraining on locally collected data. The project does not address cloud-based infrastructure monitoring, Windows endpoint agents, or encrypted traffic analysis. Compliance with specific regulatory frameworks such as Kenya's Data Protection Act (2019) is noted as relevant context but is outside the technical scope of this project.

1.5 Significance of Study

For small and medium-sized enterprises, the practical value of this project is direct. Most SMEs in Kenya and across sub-Saharan Africa currently have no structured mechanism for detecting or responding to network intrusions. A fully open-source, documented platform that can be deployed on commodity hardware lowers the barrier to entry for organisations that cannot afford commercial security tooling. The automated response capability is particularly relevant for SMEs without round-the-clock security personnel, as it reduces the window between detection and containment without requiring human intervention at the moment of an alert.

For security analysts and operations teams, the ML-driven prioritisation layer addresses the alert fatigue problem directly. By scoring and ranking events before they reach an analyst, the platform reduces the volume of alerts requiring human review and focuses attention on the events most likely to represent genuine threats. The honeypot layer adds a further layer of analyst utility: confirmed attacker sessions provide behavioural intelligence that can inform both detection rules and incident response procedures.

From an academic standpoint, this project contributes a practical implementation study of integrated threat detection combining four distinct security disciplines, SIEM, deception technology, machine learning, and security orchestration, within a single testable framework. The documented architecture, configuration procedures, and test results provide a reproducible reference for future research in this area. The project also contributes to the growing body of applied cybersecurity research emerging from African academic institutions, where practical, locally relevant security tooling remains an underrepresented area of study.

Chapter 2: Literature Review

2.1 Introduction

This chapter presents a review of the core technologies, attack categories, and existing research that underpin the Integrated Cyber Threat Detection and Response Platform developed in this project. It begins with a technology review covering the key components of the platform, including Security Information and Event Management systems, honeypot deception technology, machine learning techniques applied to intrusion detection, and Python-based security automation. It then examines the specific cyber-attack types the platform is designed to detect, analysing how each attack is executed and what network signatures it produces. Subsequently, the chapter reviews existing and proposed solutions in the literature, identifying the strengths and limitations of current approaches. The chapter concludes with a conceptual framework that illustrates how the four layers of the proposed platform interact to deliver unified threat detection and automated response.

2.2 Technology Review

This section reviews the key technologies that form the foundation of the platform, covering the SIEM platform, honeypot tool, machine learning algorithms, training datasets, automation libraries, and the virtual test environment used for development and validation.

2.2.1 Security Information and Event Management (Wazuh)

Security Information and Event Management systems centralise the collection, normalisation, and analysis of security-relevant log data generated across an organisation's network. A SIEM platform receives log data from installed agents on endpoint machines, correlates events against detection rules, and presents structured alerts through a management interface. The core advantage of a SIEM lies in its ability to aggregate events from many sources into a single location, making it possible to identify attack patterns that would not be visible from examining any individual machine in isolation (Ahmad et al., 2021).

Wazuh is an open-source SIEM platform built around four main components. The Wazuh Agent is installed on each monitored endpoint and forwards log data to the central server; it can be installed on Linux, Windows, and macOS and is configured to capture authentication logs, system call logs, file integrity events, and network connection data. The Wazuh Manager receives all incoming agent data, applies detection rules to each event, and generates alerts when rules are matched. The

Wazuh Indexer is a distributed search engine that stores all incoming events and supports historical querying. The Wazuh Dashboard provides a browser-based interface through which analysts view active alerts, search historical events, and monitor endpoint health in real time.

Wazuh was selected for this project over alternatives such as Security Onion and OSSIM because it provides native integration between all four components, requires no licensing cost, and includes an active response module that allows Python scripts to be triggered automatically when alert thresholds are reached. This active response capability is what connects the SIEM layer to the automated response layer of the platform.

2.2.2 Honeypot Deception Technology (Cowrie)

A honeypot is a deliberately exposed system or service designed to attract attackers by appearing to be a legitimate target. Unlike passive monitoring systems, honeypots generate data that is entirely composed of hostile activity: any connection to a honeypot is by definition unauthorised. This property eliminates false positives from honeypot-sourced intelligence, making it a high-confidence complement to the noisier data produced by standard log monitoring (Franco et al., 2021).

Honeypots are categorised by their level of interaction. Low-interaction honeypots emulate only the surface behaviour of a service without allowing an attacker to execute commands. Medium-interaction honeypots, such as Cowrie, present a more convincing environment that allows attackers to issue commands against a simulated shell, capturing detailed behavioural data without exposing a real operating system. Cowrie simulates SSH on port 22 and Telnet on port 23, presenting a realistic login prompt and an emulated Linux shell. When an attacker connects, Cowrie logs every authentication attempt, command issued, file download initiated, and tool dropped during the session.

All Cowrie session data is written to structured JSON log files that are forwarded to the Wazuh Manager using the Wazuh agent's custom log ingestion capability. This means honeypot events appear alongside endpoint alerts in the Wazuh Dashboard, giving analysts a unified view of both passive and deception-sourced threat intelligence. Cowrie was selected because it is actively maintained, well-documented in peer-reviewed security research, and specifically designed to capture the SSH brute force and Telnet scanning activity that this project targets.

2.2.3 Machine Learning for Intrusion Detection (Random Forest and Isolation Forest)

Machine learning approaches to intrusion detection work by modelling the statistical characteristics of normal network traffic and generating alerts when observed events deviate from that model. This is fundamentally different from rule-based detection, which can only identify attacks that match a pre-written signature. An ML-based system can detect attacks that have never been seen before, provided those attacks produce statistical patterns that differ from the normal baseline (Sarker, 2021).

Random Forest is a supervised classification algorithm that builds an ensemble of decision trees, each trained on a random subset of the training data. The final classification is determined by majority vote across all trees. This ensemble approach reduces overfitting compared to a single decision tree and handles high-dimensional feature sets effectively. Random Forest is well-suited to classifying known attack categories such as brute force, port scanning, and lateral movement because these attacks produce consistent feature signatures that labelled training data can capture. Ahmad et al. (2021) identify Random Forest as among the most reliable classifiers for network intrusion detection tasks when evaluated across benchmark datasets.

Isolation Forest is an unsupervised anomaly detection algorithm that identifies outliers by isolation rather than by modelling normal behaviour explicitly. It constructs random trees and isolates data points based on how few splits are needed to separate them from the rest of the dataset. Data points that are isolated quickly are anomalous. Isolation Forest requires no labelled training examples and is effective against threats that produce unusual but low-volume behaviour, such as slow lateral movement, periodic command and control beaconing, or abnormal outbound data volumes. By running both algorithms simultaneously, the platform can detect known attack categories through Random Forest and previously unseen or atypical threats through Isolation Forest. Both are implemented using the scikit-learn library in Python.

2.2.4 Intrusion Detection Datasets (CICIDS2017 and NSL-KDD)

Training an ML-based intrusion detection engine requires a dataset of network traffic containing both normal activity and labelled examples of specific attack types. Two publicly available benchmark datasets are used in this project. CICIDS2017 was produced by the Canadian Institute for Cybersecurity and generated using realistic traffic simulation tools. It covers brute force attempts against FTP and SSH services, denial-of-service and distributed denial-of-service attacks,

web-based attacks, network infiltration, botnet traffic, and port scans. The dataset provides network flow features including packet counts, flow duration, inter-arrival times, and flag counts, which serve as input features for the ML models. Khraisat and Alazab (2021) identify CICIDS2017 as one of the most representative publicly available datasets for evaluating modern intrusion detection approaches.

NSL-KDD is a refined version of the KDD Cup 1999 dataset that addresses known issues with duplicate records and class imbalance present in the original. It covers four broad attack families: denial-of-service, probe (reconnaissance), remote-to-local, and user-to-root attacks. NSL-KDD is included alongside CICIDS2017 to broaden detection coverage, particularly for reconnaissance and privilege escalation categories that CICIDS2017 does not fully address. The two datasets are preprocessed separately, normalised using min-max scaling, and their common features are aligned before the combined dataset is used for model training.

2.2.5 Python-Based Security Automation and Orchestration

Security Orchestration, Automation and Response (SOAR) refers to the practice of automating the execution of response actions when security events occur, reducing the time between threat detection and containment. Commercial SOAR platforms exist but typically require enterprise licensing that puts them out of reach for small organisations. This project replicates core SOAR functionality using Python scripts that integrate directly with Wazuh's built-in active response framework, achieving automated containment without commercial tooling.

Wazuh active response allows administrators to define scripts that are triggered automatically when alerts matching defined criteria are generated. The platform uses this mechanism to connect detection output to three containment playbooks. The first executes an iptables firewall rule to block all traffic from an attacking IP address, using Python's subprocess library to issue the system-level command. The second locks a flagged user account using the passwd --lock command. The third sends an email notification to the administrator using Python's smtplib library, including the event details, the ML classification result, and the response action taken. Together these playbooks address the most immediate containment needs following threat detection without requiring manual intervention.

2.2.6 Virtualisation and Attack Simulation (VMware and Kali Linux)

Virtualisation allows multiple isolated operating systems to run simultaneously on a single physical host machine. VMware Workstation is used in this project to create and manage the five virtual machines that form the test environment. VMware Workstation is selected because it is open-source, cross-platform, and supports internal network adapter configurations that isolate test traffic from any external network, allowing attacks to be simulated safely without risk to external systems.

The five virtual machines serve distinct roles. The Wazuh server VM runs Ubuntu Server 22.04 LTS and hosts the Wazuh Manager, Indexer, and Dashboard. Two endpoint VMs, also running Ubuntu Server 22.04 LTS, have Wazuh agents installed and represent the monitored machines that attacks target. The Cowrie honeypot runs as a standalone VM connected to the same internal network. The Kali Linux VM serves as the attack machine from which all simulated attacks are launched. Kali Linux is a Debian-based operating system distributed specifically for security testing and includes a comprehensive set of pre-installed tools covering the full range of attack simulation needed for this project: Hydra for brute force and credential stuffing, Nmap for port scanning and reconnaissance, Metasploit for privilege escalation and lateral movement, and custom Python scripts for data exfiltration and command and control beaconing simulation.

2.3 Cyber Attack Vulnerability Analysis

This section examines the seven attack categories the platform is designed to detect. For each type, the analysis covers how the attack is executed, what vulnerabilities it exploits, and what log-level or network-level signatures it produces that the ML engine and Wazuh detection rules are trained to identify.

2.3.1 Brute Force and Credential Stuffing Attacks

Brute force attacks systematically attempt username and password combinations against an authentication service until a valid pair is found. Against SSH services, this typically involves a tool such as Hydra generating thousands of login attempts from a wordlist in rapid succession, targeting common usernames such as root, admin, and ubuntu. Credential stuffing extends this by using databases of username and password pairs leaked from previous data breaches, reasoning that users frequently reuse credentials across different services and platforms.

Both techniques exploit the absence of account lockout policies or rate limiting on authentication services. SSH is a particularly common target because it is widely deployed and frequently exposed on its default port without additional access controls. The network and log signatures are consistent: a single source IP generates a high volume of failed authentication events against one or more usernames within a short time window, visible in /var/log/auth.log on Linux systems. Wazuh rule group 5700 monitors SSH authentication events, triggering when failed login attempts from a single source exceed a configured threshold within a defined interval. The Cowrie honeypot captures all SSH brute force attempts directed at its IP address, logging the credentials attempted, the tools used, and any commands executed following a simulated successful login.

2.3.2 Port Scanning and Network Reconnaissance

Port scanning maps the open services on a target host or network to identify potential entry points before a more targeted attack is launched. Nmap supports a range of scan types including TCP SYN scans, which send connection requests without completing the three-way handshake, UDP scans, and version detection scans that identify the software and version running on each open port. Host discovery techniques such as ping sweeps identify which machines are active on a subnet before individual ports are probed.

Port scanning exploits the fundamental operation of TCP/IP networking: services listening on open ports respond to connection requests, revealing their presence. The network-level signature is distinctive a single source IP contacts a large number of destination ports on one or more hosts within a short time window. In packet-level data this appears as a burst of SYN packets to sequential or random port numbers. Wazuh's integration with network monitoring captures this pattern, and the ML engine uses source-to-destination port diversity and connection rate as key classification features. Port scanning is most commonly a precursor to further attacks, making its detection significant as an early warning indicator.

2.3.3 Lateral Movement

Lateral movement refers to techniques used by an attacker who has already compromised one host to extend access to other machines on the same network. Common methods include using stolen credentials to authenticate to adjacent hosts via SSH, exploiting shared network services, or leveraging unpatched vulnerabilities on neighbouring systems. The goal is typically to reach a

higher-value target such as a database server, a file server containing sensitive data, or a system with elevated privileges.

Lateral movement is particularly difficult to detect because it uses legitimate protocols and valid credentials and does not generate the high-volume signatures associated with brute force or denial-of-service attacks. The key indicator is a change in an internal host's communication behaviour: a machine that normally connects to a small set of known internal services suddenly begins initiating connections to many other internal machines it has no prior communication history with. This deviation from an established baseline is precisely what the Isolation Forest component of the ML engine is designed to detect. Because Isolation Forest identifies anomalies relative to normal behaviour rather than matching known signatures, it is better suited to catching slow, deliberate lateral movement that stays below alert thresholds.

2.3.4 Privilege Escalation

Privilege escalation allows an attacker or process operating with limited user rights to gain elevated system permissions, typically root or administrator level access. On Linux systems, common methods include exploiting misconfigured sudo permissions that allow a low-privileged user to run specific commands as root, abusing SUID binaries that run with the permissions of their owner regardless of who executes them, and exploiting kernel vulnerabilities present on unpatched systems.

Log-level signatures of privilege escalation are visible in several places. Unexpected sudo command usage by accounts that do not normally use sudo appears in /var/log/auth.log and is captured by Wazuh agent log collection. Modifications to sensitive system files such as /etc/passwd, /etc/shadow, or /etc/sudoers trigger Wazuh's file integrity monitoring module, which generates alerts when these files change outside of expected maintenance windows. The sudden execution of privileged processes by accounts not normally associated with administrative activity is an additional indicator that the ML engine is trained to recognise from system call log features.

2.3.5 Data Exfiltration

Data exfiltration involves the unauthorised transfer of sensitive data from a compromised host to an external location controlled by the attacker. Attackers commonly use standard protocols such as HTTP, FTP, and DNS to carry exfiltrated data, as this traffic blends into normal network activity

and is less likely to be blocked by firewall rules. DNS tunnelling, where data is encoded into DNS query strings sent to an attacker-controlled domain, is a particularly covert method because DNS traffic is frequently permitted through corporate firewalls without deep inspection.

The network-level signatures of data exfiltration include an abnormal volume of outbound traffic from a host, large data transfers to external IP addresses the host has no prior relationship with, and in the case of DNS tunnelling, an unusually high rate of DNS queries containing long and unusual subdomain strings. These deviations from a host's normal outbound traffic baseline are what the Isolation Forest model flags. A fixed-threshold rule-based approach is less effective here because exfiltration volume and destination vary between incidents; behavioural baseline anomaly detection adapts to each host's individual traffic profile.

2.3.6 Command and Control Beaconing

Command and control beaconing occurs when malware installed on a compromised host makes regular, periodic outbound connections to an attacker-controlled server to receive instructions, report status, or exfiltrate data in small increments. The defining characteristic of beaconing is regularity: the host contacts the same external address at consistent intervals, typically ranging from minutes to hours depending on the malware configuration.

This regularity makes beaconing detectable even though individual connections may appear indistinguishable from legitimate web traffic. The Isolation Forest component is sensitive to the statistical regularity of connection timing because this pattern is anomalous relative to the irregular intervals of normal user-driven traffic. Additional indicators include consistent packet sizes per connection, connections to external IPs with no prior relationship to the host, and the absence of any interactive user session associated with the connection events. Signature-based systems that rely on blocklists of known command and control server addresses miss beaconing from newly registered or dynamically generated domains, which is a common evasion technique. The behavioural anomaly approach used in this project does not require prior knowledge of the command and control server's identity.

2.3.7 Suspicious Authentication Patterns

Beyond the high-volume signature of a brute force attack, lower-volume authentication anomalies can indicate account compromise or insider threat activity. These patterns involve successful authentication events that individually do not trigger rule-based detection but are anomalous relative to the account's established historical behaviour (Vielberth et al., 2021).

The patterns targeted by this platform include: successful logins at hours outside an account's normal usage window, such as late-night or early-morning access for an account whose history shows only business-hours activity; logins originating from IP addresses not previously associated with a given account; and simultaneous active sessions from two different source addresses for the same account, which is a strong indicator of credential theft. Wazuh captures authentication events including login timestamps and source IP addresses from Linux system logs. The ML engine uses historical authentication data to establish a per-account baseline and flags deviations, making this detection particularly relevant to post-compromise account use and insider threat scenarios where an attacker operates with valid credentials but in ways that differ from the legitimate user's normal behaviour.

2.4 Existing Solutions for Integrated Threat Detection

This section reviews existing research relevant to the platform's four layers, examining the approach, findings, and limitations of each study. The purpose is to establish what prior work has achieved and to identify the specific gap that this project addresses.

Table 2.1: Comparison of Reviewed Studies

Study	**Approach & Tools**	**Key Finding**	**Gap This Project Addresses**
Sarker (2021)	Survey of ML algorithms including Random Forest and Isolation Forest across cybersecurity domains	Ensemble methods most effective for threat classification; anomaly detection captures novel threats	No operational integration with a live SIEM data stream or automated response system

Franco et al. (2021)	Systematic survey of honeypot and honeynet deployments in IoT and cyber-physical environments using tools such as Cowrie	Medium-interaction honeypots produce higher-confidence threat intelligence than passive monitoring	No integration of honeypot intelligence with a SIEM pipeline or ML detection engine
Ahmad et al. (2021)	Comparative evaluation of ML and deep learning classifiers on CICIDS2017 and NSL-KDD datasets	Random Forest most consistent for multi-class intrusion detection across both datasets	Evaluation conducted entirely offline; no real-time deployment or automated response capability tested

2.4.1 Machine Learning: Algorithms, Real-World Applications and Research Directions

Sarker (2021) presents a comprehensive survey of machine learning algorithms and their applications across a range of domains, with a specific focus on cybersecurity. The paper categorises supervised, unsupervised, and reinforcement learning approaches and evaluates their suitability for threat detection tasks. Key findings include the consistent high performance of ensemble methods such as Random Forest for network classification tasks and the growing role of unsupervised anomaly detection in catching threats that evade labelled-data classifiers. Sarker's work provides the theoretical foundation for the dual-algorithm ML engine used in this project.

The limitation of Sarker's survey relative to this project is that it evaluates algorithms in isolation and does not address how ML detection can be operationally integrated with a live SIEM data stream or connected to automated response systems. This project bridges that gap by embedding the ML engine directly within the Wazuh event pipeline and connecting its output to automated containment playbooks.

2.4.2 A Survey of Honeypots and Honeynets for IoT and Cyber-Physical Systems

Franco et al. (2021) conduct a systematic survey of honeypot and honeynet deployments across IoT and industrial cyber-physical system environments, documenting the design, deployment, and data collection methodologies used across a range of studies. The survey finds that medium-interaction honeypots, such as Cowrie, consistently provide higher-quality behavioural intelligence than low-interaction alternatives while maintaining an acceptable risk profile for use in operational environments. Franco et al. also confirms that honeypot-sourced data significantly reduces false positive rates in combined detection systems compared to traffic-based monitoring alone.

The limitation of the surveyed work related to this project is that none of the deployments reviewed integrates honeypot intelligence with a SIEM or ML detection pipeline. Honeypot data is collected and analysed offline in most cases, with no mechanism for feeding attacker profiles into an active detection or response system. This project addresses that gap directly by configuring Cowrie to forward session logs into the Wazuh pipeline in real time, making honeypot intelligence immediately actionable.

2.4.3 Network Intrusion Detection: A Systematic Study of Machine Learning and Deep Learning Approaches

Ahmad et al. (2021) evaluate a broad range of machine learning and deep learning classifiers against multiple intrusion detection benchmark datasets, including CICIDS2017 and NSL-KDD. The study provides comparative performance metrics across classifiers and datasets and identifies Random Forest as the most consistently high-performing algorithm for multi-class intrusion detection when balanced against computational overhead. The paper also identifies the absence of real-time, operationally integrated testing as a common limitation across the reviewed studies: most evaluations are conducted entirely offline using pre-labelled datasets.

This limitation is directly relevant to this project. By integrating the trained ML model with live Wazuh event data rather than evaluating it offline, this project takes a step toward operational validation that the reviewed literature does not address. The performance metrics from Ahmad et al. (2021) also inform the benchmark targets defined in this project's non-functional requirements, specifically the 85 percent minimum detection accuracy target.

2.4.4 Existing Research Gap

The literature reviewed across sections 2.4.1 to 2.4.3 demonstrates that SIEM-based monitoring, honeypot deception technology, and machine learning intrusion detection have each been studied individually and shown to be effective within their respective domains. However, no reviewed study integrates all three approaches alongside automated incident response into a single, operationally tested platform. Sarker (2021) and Ahmad et al. (2021) evaluate ML algorithms without live SIEM integration; Franco et al. (2021) document honeypot deployments without connecting them to detection or response pipelines. The absence of an open-source, integrated platform that combines these four capabilities and does so within the resource constraints of a small or medium-sized enterprise is the specific gap that this project addresses.

2.5 Conceptual Framework

The conceptual framework presented in this section illustrates the operational flow of the integrated platform, showing how the four layers interact to deliver unified threat detection and automated response. The framework is structured as a data flow model in which events move from generation at monitored endpoints, through detection and classification, to automated containment.

Network events and system logs generated on monitored Linux endpoints are collected continuously by installed Wazuh agents and forwarded to the Wazuh Manager. Concurrently, any connection attempts made to the Cowrie honeypot are logged in structured JSON format and forwarded to the same Wazuh Manager via a custom log source configuration. At the Wazuh Manager, all incoming events from both endpoint agents and the honeypot are normalised into a common format, evaluated against Wazuh's built-in rule set, and stored in the Wazuh Indexer.

The ML engine operates as a continuous listener on the Wazuh event stream. Each incoming event is passed through a feature extraction pipeline that transforms the raw log data into the numerical feature vectors expected by the trained models. The Random Forest classifier assigns a multi-class label indicating the most likely attack category, while the Isolation Forest model assigns an anomaly score reflecting how far the event deviates from the normal baseline. If either model produces a result meeting the critical severity threshold, the Wazuh active response module is triggered and passes the event to the Python automated response module.

The automated response module selects and executes the appropriate playbook based on the event type: IP blocking for external attack sources, account isolation for compromised user credentials, or administrator notification for events requiring human review. All events, ML classifications, and response actions are recorded and displayed in the Wazuh Dashboard, giving the analyst a complete, real-time picture of network security status. This framework forms the basis for the system design and implementation described in Chapter 3.

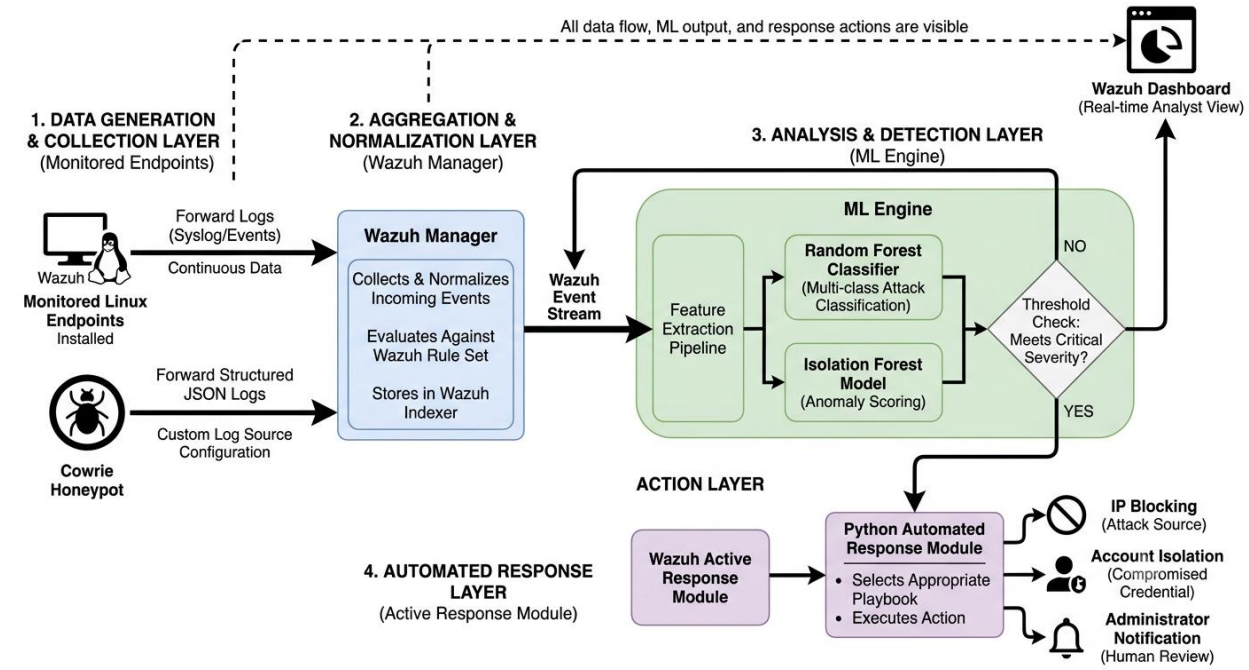


Figure 2.5: Conceptual Framework of the integrated platform

Chapter 3: Design Methodology

3.1 Introduction

This chapter describes the approach taken to design and build the Integrated Cyber Threat Detection and Response Platform. It covers the overall project methodology, the functional and non-functional requirements the system must satisfy, and the hardware and software needed to implement and test it. Together these sections define what the system must do, how it will be built, and what resources are required to deliver it within the project timeline.

3.2 Project Methodology

This project adopts an Agile Design Science Research (Agile DSR) methodology. Agile DSR combines the rigour of Design Science Research, which demands that a project produce a testable, evaluated artefact rather than a purely theoretical output with the iterative, sprint-based structure of Agile development. This combination is appropriate for this project because the platform is complex enough to benefit from incremental construction and testing, and requirements for later layers (such as the ML engine) only become fully clear once earlier layers (such as the SIEM core) are working.

The project is structured into four iterative sprints, each following a plan, design, build, and test cycle. Sprint 1 covers SIEM deployment: the Wazuh server is configured, agents are installed on endpoint virtual machines, and baseline log collection and alerting are validated. Sprint 2 covers honeypot integration: Cowrie is deployed and configured to forward session logs into the Wazuh pipeline, then tested against simulated SSH scanning activity. Sprint 3 covers the ML engine: the Random Forest and Isolation Forest models are trained on CICIDS2017 and NSL-KDD, integrated with the live Wazuh event stream, and validated against labelled test data. Sprint 4 covers automated response and end-to-end validation: the Python playbooks are connected to Wazuh's active response framework, and the complete platform is tested against five simulated attack scenarios using Kali Linux. Findings from each sprint inform the next, allowing the architecture to be refined progressively.

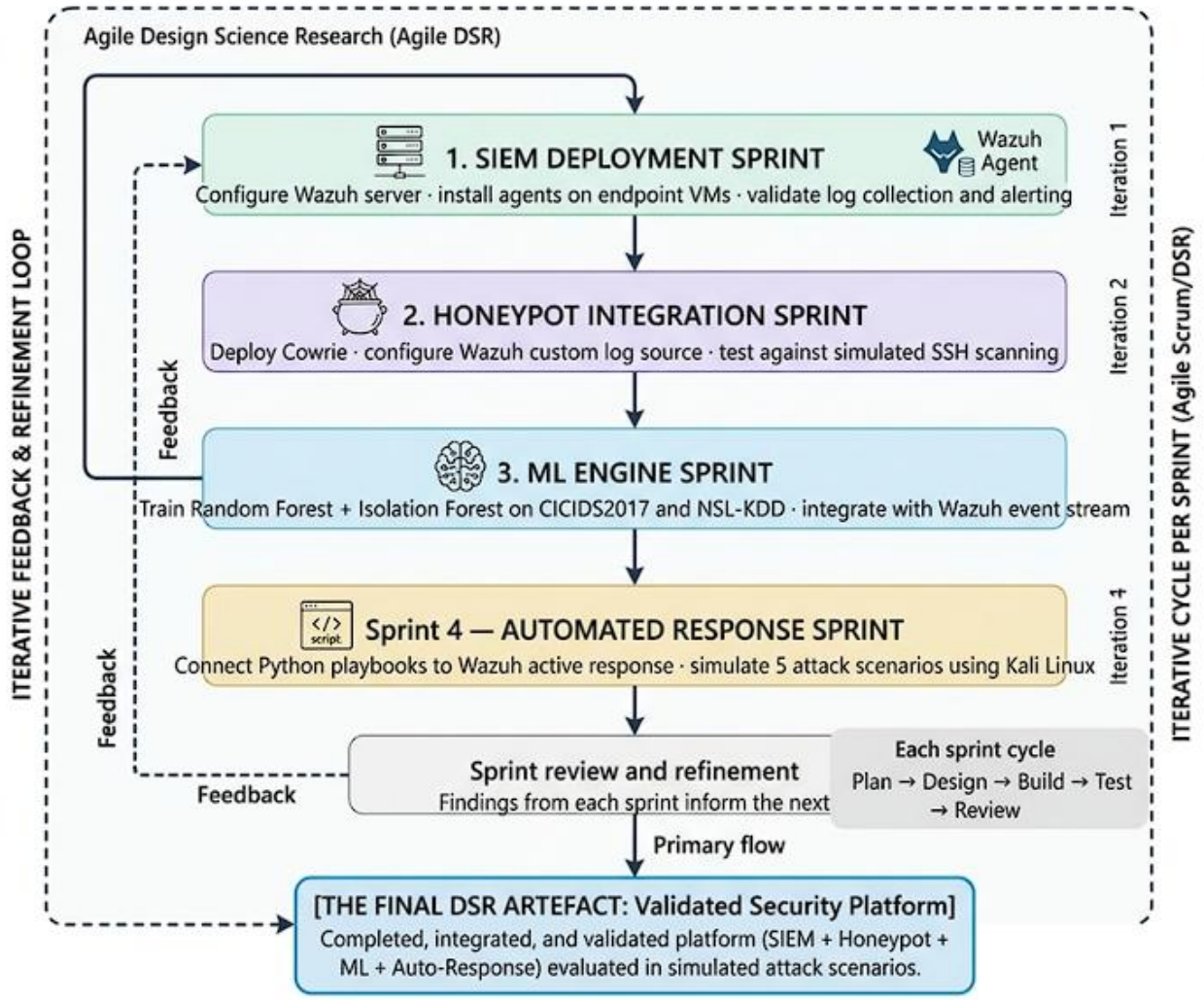


Figure 3.1: Agile DSR Methodology – Project Sprint Structure

3.3 Feature and Design Requirements

3.3.1 Functional Requirements

The platform must collect and normalise log data from a minimum of three Linux endpoint machines to a central Wazuh Manager through installed Wazuh agents. This covers authentication logs, system call logs, network connection events, and file integrity monitoring data. The Wazuh Manager must process incoming events in real time, apply detection rules, and generate structured alerts for events that match known threat patterns. Alert severity must be assigned on a scale that allows the platform to distinguish between low, medium, and critical events.

The Cowrie honeypot must run as a medium-interaction SSH and Telnet service on the internal network segment and log all connection attempts, authentication credentials submitted, and commands executed during attacker sessions. All Cowrie session logs must be forwarded automatically to the Wazuh Manager as a custom log source so that honeypot events appear alongside endpoint alerts in the unified dashboard.

The machine learning engine must receive structured event data from the Wazuh pipeline, extract the relevant features, and classify each event as normal, suspicious, or critical using the trained Random Forest and Isolation Forest models. Classification results must be passed back into the Wazuh alerting system so that ML-generated severity scores are visible on the Wazuh Dashboard alongside rule-based alerts.

Upon a threat being classified as critical or high severity, the automated response module must execute the appropriate response playbook within sixty seconds of alert generation. The three playbooks are: firewall-based IP blocking using iptables rules, user account isolation using system-level account lock commands, and email notification to the designated administrator. The Wazuh Dashboard must provide a unified real-time view of active alerts, ML classification results, honeypot session activity, and response actions that have been executed.

3.3.2 Non-Functional Requirements

The machine learning engine must achieve a minimum detection accuracy of 85 percent across all seven attack categories on the test dataset. The false positive rate must not exceed 15 percent, keeping the platform operationally manageable for a small security team. Mean time to detect (MTTD), measured from the moment an attack event is generated to the moment a critical alert appears in the Wazuh Dashboard, must be under 30 seconds. Mean time to respond (MTTR), measured from alert generation to the execution of a containment action by the automated response module, must be under 60 seconds.

The complete platform must be deployable on a single host machine running VirtualBox, with a minimum of 8 GB RAM and 500 GB of available disk storage. This reflects the project's goal of providing a solution accessible to organisations with limited infrastructure. All components must be open-source and require no licensing cost. The system must operate without internet connectivity during testing, relying entirely on the internal virtual network.

3.4 Implementation Requirements

3.4.1 Hardware Requirements

A single host machine is used to run the complete virtual test environment. The host must have a minimum of 8 GB RAM to support the simultaneous operation of five virtual machines: the Wazuh server, two Linux endpoint machines with Wazuh agents installed, the Cowrie honeypot instance, and the Kali Linux attack machine. A minimum of eight processor cores is required to maintain stable concurrent VM operation. At least 500 GB of available disk storage is needed for VM images, Wazuh log indexes, and the ML training dataset files. The host machine must include a network interface that supports VirtualBox internal network adapters so that inter-VM communication is isolated from external networks during testing.

3.4.2 Software Requirements

Two categories of software are used in this project. Platform software includes VMware Workstation for virtual machine management; Ubuntu Server 22.04 LTS for the Wazuh server VM and monitored endpoint VMs; Wazuh 4.14.x, which provides the Wazuh Manager, Wazuh Indexer, and Wazuh Dashboard components in a single deployable package; and Cowrie for SSH and Telnet honeypot emulation. Ubuntu Server 22.04 LTS is selected because it is a long-term support release

with wide compatibility with both Wazuh agents and Cowrie, reducing configuration friction during integration.

Development and testing software includes Python 3.10 or later with the scikit-learn, pandas, numpy, and joblib libraries for ML model training and real-time inference; iptables for firewall-based automated IP blocking in the response module; and Kali Linux 2025.x for attack simulation. Kali Linux is selected because it provides a comprehensive set of pre-installed security testing tools in a single environment. The specific tools used for attack simulation are Hydra for brute force and credential stuffing, Nmap for port scanning and network reconnaissance, Metasploit for privilege escalation and lateral movement scenarios, and custom Python scripts for data exfiltration and command and control beaconing simulation. All listed software is open-source and freely available.

3.5 Testing and Validation

This section describes how the integrated platform is tested to confirm that it meets the functional and non-functional requirements defined in Section 3.3. Testing is carried out in stages, moving from individual component checks to full end-to-end validation of the complete platform under simulated attack conditions.

3.5.1 Testing Strategy

Testing follows three levels. Unit-level testing verifies that each layer functions correctly in isolation: the Wazuh agents are confirmed to forward logs correctly, the Cowrie honeypot is confirmed to capture session data accurately, the ML models are evaluated against held-out test data from CICIDS2017 and NSL-KDD before live integration, and each Python response playbook is tested independently to confirm it executes its intended action. Integration testing follows, verifying that data flows correctly between layers, for example, confirming that Cowrie logs reach the Wazuh Manager, and that ML classification results successfully trigger the automated response module. System-level testing is performed last, evaluating the complete platform as a single unit under simulated attack conditions that mirror real-world usage.

3.5.2 Test Environment

All testing is conducted within the VirtualBox environment described in Section 3.4, consisting of the Wazuh server, two monitored Linux endpoints, the Cowrie honeypot, and the Kali Linux attack machine, all connected through an isolated internal network adapter. Conducting tests in this isolated environment ensures that simulated attacks cannot affect external systems and that results are reproducible across repeated test runs.

3.5.3 Attack Simulation Scenarios

Five attack scenarios are executed from the Kali Linux VM to validate detection and response across the seven target attack categories defined in Section 1.4. Scenario one combines brute force and credential stuffing, using Hydra to launch a dictionary-based SSH login attack against both a monitored endpoint and the Cowrie honeypot. Scenario two covers port scanning and network reconnaissance, using Nmap to perform a SYN scan and service version detection against the internal network segment. Scenario three simulates lateral movement, using a compromised endpoint to initiate SSH connections to multiple other internal hosts in a pattern inconsistent with its normal communication baseline. Scenario four covers privilege escalation, executing a known sudo misconfiguration exploit using Metasploit against a deliberately vulnerable endpoint configuration. Scenario five combines data exfiltration and command and control beaconing, using a custom Python script to transfer a test file to an external address while generating periodic outbound connections at fixed intervals to simulate beaconing behaviour. Each scenario is executed three times to confirm consistency of detection results.

3.5.4 Validation Metrics and Acceptance Criteria

Platform performance is measured against the non-functional requirements defined in Section 3.3.2. Detection accuracy is calculated as the proportion of simulated attack events correctly classified by the ML engine, with a minimum acceptance threshold of 85 percent across the seven attack categories. False positive rate is calculated by introducing a baseline of normal, non-malicious traffic alongside the attack scenarios and measuring the proportion incorrectly flagged as malicious, with a maximum acceptable rate of 15 percent. Mean time to detect is recorded as the interval between attack execution and the appearance of a critical alert on the Wazuh Dashboard, with a target of under 30 seconds. Mean time to respond is recorded as the interval between alert generation and the execution of the corresponding containment action, with a target of under 60 seconds. Results from all five scenarios are tabulated and compared against these

benchmarks to determine whether the platform meets its design requirements, with any scenario falling outside the acceptance criteria flagged for further tuning of detection rules or model parameters.

References

Ahmad, Z., Khan, A. S., Shiang, C. W., Abdullah, J., & Ahmad, F. (2021). Network intrusion detection system: A systematic study of machine learning and deep learning approaches. Transactions on Emerging Telecommunications Technologies, 32(1), e4150. https://doi.org/10.1002/ett.4150

Franco, J., Aris, A., Canberk, B., & Uluagac, A. S. (2021). A survey of honeypots and honeynets for internet of things, industrial internet of things, and cyber-physical systems. IEEE Communications Surveys & Tutorials, 23(4), 2351–2383. https://doi.org/10.1109/COMST.2021.3106669

Khraisat, A., & Alazab, A. (2021). A critical review of intrusion detection systems in the internet of things: Techniques, deployment strategy, taxonomy, datasets and future directions. Electronics, 10(21), 2633. https://doi.org/10.3390/electronics10212633

Sarker, I. H. (2021). Machine learning: Algorithms, real-world applications and research directions. SN Computer Science, 2(3), 160. https://doi.org/10.1007/s42979-021-00592-x

https://www.researchgate.net/publication/350297716_Machine_Learning_Algorithms_Real-World_Applications_and_Research_Directions

Vielberth, M., Böhm, F., Fichtinger, I., & Pernul, G. (2021). Security operations center: A systematic study and open challenges. IEEE Access, 9, 119023–2119042. https://ui.adsabs.harvard.edu/link_gateway/2020IEEEA...8v7756V/doi:10.1109/ACCESS.2020.3045514